

密码算法工程实践 · 作业 4

SM3 软件实现与优化

基于 SIMD 寄存器与通用寄存器的混合优化

—— ARM64 NEON 与 x86 AVX2 / AVX-512 双指令集架构实践

成员姓名
彭炜霖
李庭跃
张 涵

66C7F0F4 62EEEDD9 D1F2D46B DC10E4E2

4167C487 5CF2F7A2 297DA02B 8F4BA8E0

DEBE9FF9 2275B8A1 38604889 C18E5A4D

2026 年 7 月

目 录

1 实验任务与环境	1
1.1 实验任务	1
1.2 实验环境	1
2 SM3 算法原理	1
2.1 算法总体流程	1
2.2 消息扩展与迭代压缩	1
3 混合优化设计	1
3.1 可并行性分析与混合策略	1
3.2 混合优化总体架构	1
3.3 消息扩展依赖分析与向量化	1
3.4 多块并行 multi-buffer	1
3.5 运行时调度与安全回退	1
4 关键实现	1
4.1 工程结构与构建	1
4.2 ARM64 NEON 实现要点	1
4.3 x86 AVX2 / AVX-512 实现要点	1
5 正确性验证	1
5.1 标准测试向量	1

5.2 跨实现一致性验证	1
6 性能测试与分析	1
6.1 测试方法	1
6.2 x86-64 平台结果	1
6.3 ARM64 平台结果	1
6.4 结果分析	1
7 结论与体会	1
参考文献	1

1 实验任务与环境

1.1 实验任务

本实验依据 GB/T 32905-2016 《信息安全技术 SM3 密码杂凑算法》完成 SM3 的软件实现与性能优化。任务要求实现一种基于 SIMD 寄存器与通用寄存器混合的 SM3 优化算法，并覆盖 ARM64 与 x86 (AVX2 / AVX-512) 两种指令集架构，最终交付可正常构建、运行、验证的工程代码包与本实验报告。

围绕该任务，本工程实现了四个层次的代码：纯通用寄存器的标量参考实现（正确性基准与性能基线）、单消息流的“SIMD 消息扩展 + 通用寄存器压缩”混合实现（ARM64 NEON 与 x86 SSSE3 两个版本）、基于宽 SIMD 寄存器的多块并行实现（NEON 4 路、AVX2 8 路、AVX-512 十六路），以及统一的运行时调度层。

1.2 实验环境

全部性能数据均在表 1 所列环境中实际运行测得。x86-64 平台直接在本机运行；ARM64 平台通过交叉编译生成目标代码，并以 qemu-user 模式翻译执行，其绝对吞吐率不代表真实 ARM 硬件，但同一环境下的相对加速比具有参考价值。

项目	配置
处理器	Intel(R) Xeon(R) Platinum (2 vCPU, 支持 AVX2 / AVX-512F / BW / VL)
内存 / 缓存	4 GiB / L1d 32 KiB、L2 1 MiB
操作系统	Debian GNU/Linux 12 (bookworm) , x86_64
本机编译器	GCC 12.2.0, -O3
ARM64 交叉编译	Arm GNU Toolchain 12.2.Rel1 (aarch64-none-linux-gnu-gcc 12.2.1)
ARM64 运行环境	qemu-aarch64-static 7.2.22 (qemu-user 模式)

表 1 实验软硬件环境

2 SM3 算法原理

2.1 算法总体流程

SM3 是我国商用密码杂凑算法标准，输出 256 bit 摘要，广泛应用于数字签名、完整性校验与随机数生成等场景。算法整体采用 Merkle–Damgård 迭代结构：消息经 MD 强化填充后按 512 bit 分组，逐组与迭代寄存器 V 一同进入压缩函数 CF，最终迭代值即为杂凑值，流程如图 1 所示。

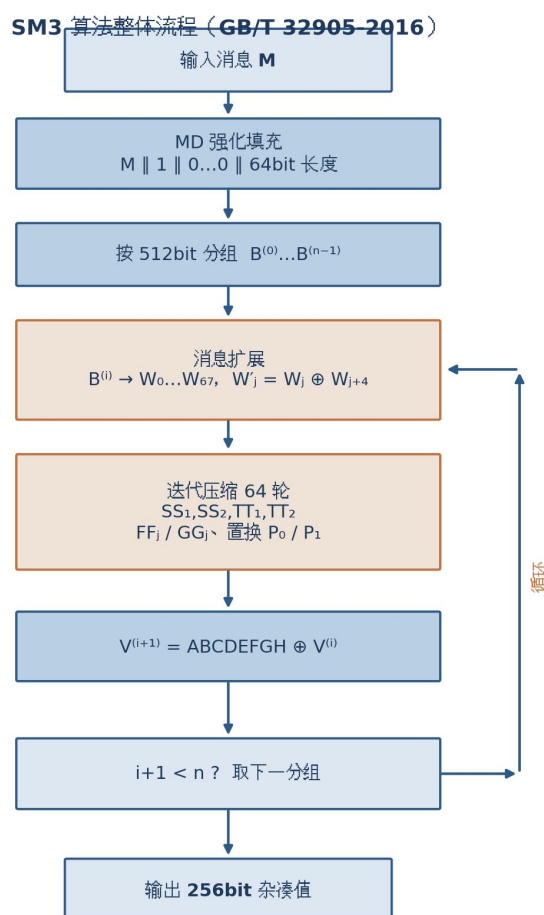


图 1 SM3 算法整体流程

2.2 消息扩展与迭代压缩

每个 512 bit 分组先按大端序拆为 16 个 32 bit 字 $W[0..15]$ ，再递推扩展出 52 个新字；压缩阶段同时使用 $W[j]$ 与 $W'[j] = W[j] \oplus W[j+4]$ 。核心递推与轮函数如下：

```
W[j] = P1( W[j-16] ⊕ W[j-9] ⊕ ROTL(W[j-3], 15) ) ⊕ ROTL(W[j-13], 7) ⊕ W[j-6]
SS1 = ROTL( ROTL(A,12) + E + ROTL(Tj, j mod 32), 7 );    SS2 = SS1 ⊕ ROTL(A,12)
TT1 = FFj(A,B,C) + D + SS2 + W'[j];    TT2 = GGj(E,F,G) + H + SS1 + W[j]
```

其中 $P0(X) = X \oplus \text{ROTL}(X,9) \oplus \text{ROTL}(X,17)$, $P1(X) = X \oplus \text{ROTL}(X,15) \oplus \text{ROTL}(X,23)$ ；前 16 轮 FF、GG 均为逐位异或，后 48 轮 FF 为多数函数（maj）、GG 为选择函数（choice）。64 轮迭代后，ABCDE FGH 与输入迭代值异或得到新的 V。可以看出，压缩阶段的 8 个状态字 A-H 构成一条 64 级的串行依赖链，这是后续混合优化设计的出发点。

3 混合优化设计

3.1 可并行性分析与混合策略

对 SM3 做 SIMD 优化，先要回答“哪些部分适合 SIMD、哪些必须留在通用寄存器”。本实验从三个粒度分析其可并行性：

(1) 迭代压缩（串行）。压缩轮中 A-H 每轮严格依赖上一轮结果，单条消息内无法数据级并行，强行向量化只会引入大量跨 lane 洗牌开销。这部分保留在通用寄存器上执行，反而能利用标量流水线的短延迟。

(2) 消息扩展（部分并行）。 $W[j]$ 的递推中， $W[j-16]$ 、 $W[j-9]$ 、 $W[j-13]$ 、 $W[j-6]$ 与当前计算点的距离均不小于 6，可提前批量取用；只有 $W[j-3]$ 是近距依赖，成为向量化瓶颈。可通过“通用寄存器先行求解链头、SIMD 批量完成其余”的混合方式化解（见 3.3 节）。

(3) 消息块之间（完全并行）。多条独立消息的哈希计算互不依赖，可将 N 条消息分别放入 SIMD 寄存器的 N 个 32 bit lane 中，扩展与压缩全程逐 lane 执行，即 multi-buffer 多块并行。

由此形成“单流混合 + 多块并行”的两层混合优化策略：单条消息内让 SIMD 寄存器与通用寄存器各司其职；面对批量消息时再叠加块间数据级并行，把寄存器宽度用满。

3.2 混合优化总体架构

工程总体架构如图 2 所示。层次一为单消息流块内混合：消息扩展由 SIMD 寄存器完成（含大端字节序转换、P1 置换、循环移位与异或），64 轮压缩链由通用寄存器完成；层次二为多块并行：N 条等长消息经转置装载到宽 SIMD 寄存器后全程并行处理。两层共享同一套对外 API，由运行时调度层按 CPU 能力自动选择实现并无条件回退，保证工程在任何目标机器上均可正确运行。

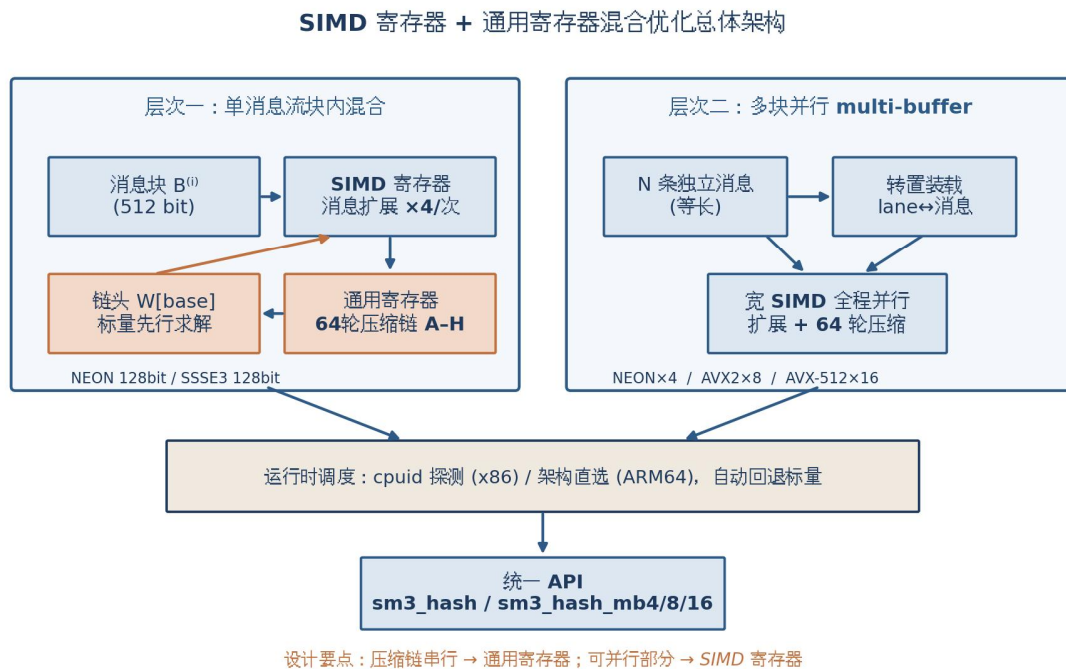


图 2 SIMD 寄存器 + 通用寄存器混合优化总体架构

3.3 消息扩展依赖分析与向量化

消息扩展的递推依赖如图 3 所示。以 4 个字为一组考察 $W[\text{base}..\text{base}+3]$ ：计算 $W[\text{base}]$ 所需的全部输入均为已知旧值；而 $W[\text{base}+3]$ 需要用到同组的 $W[\text{base}]$ 。因此每组只需先用通用寄存器

按递推式求出链头 $W[\text{base}]$ ，随后 SIMD 寄存器以 $W[\text{base}-16..\text{base}-13]$ 、 $W[\text{base}-9..\text{base}-6]$ 、 $W[\text{base}-13..\text{base}-10]$ 、 $W[\text{base}-6..\text{base}-3]$ 与向量 $[W[\text{base}-3], W[\text{base}-2], W[\text{base}-1], W[\text{base}]]$ 为输入，一次性算出 4 个新字，其中 lane 0 的结果与链头一致（自然覆盖）。这样既打破了跨 lane 的近距离依赖，又把 P1、 $\text{ROTL}(\cdot, 15)$ 、 $\text{ROTL}(\cdot, 7)$ 和全部异或操作向量化，是典型的“标量解链 + SIMD 批量”混合计算。

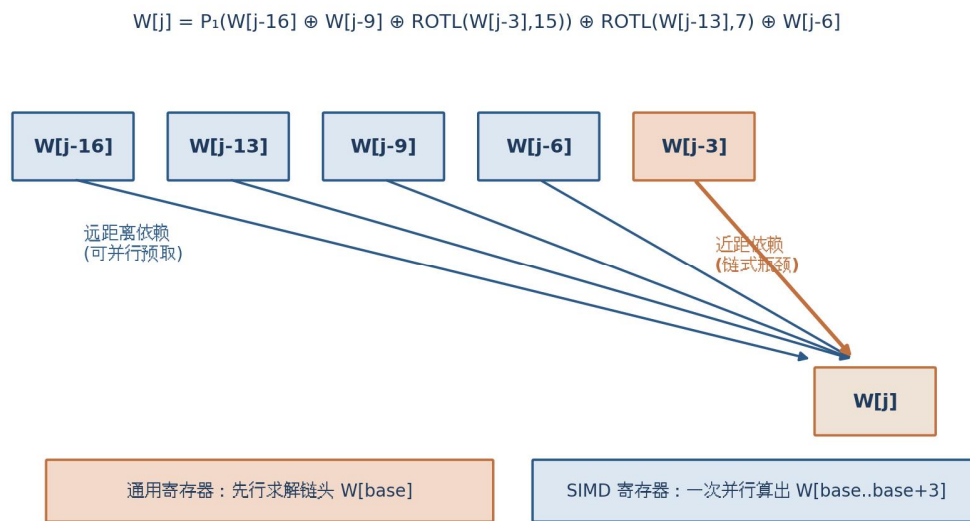


图 3 消息扩展依赖分析与混合向量化方案

3.4 多块并行 multi-buffer

多块并行将 N 条独立消息的第 b 个分组按字转置：向量 $W[j]$ 的 lane i 存放第 i 条消息的 $W[j]$ 。转置后，每条消息在自己的 lane 内独立演化，消息扩展不再存在跨 lane 依赖，可完全向量化；64 轮压缩中的 FF、GG、P0、P1、 ROTL 与模加也全部替换为等价的 SIMD 逐 lane 指令。各消息独立做 MD 强化填充后逐组处理，最后逆转置写出 N 份摘要。本工程在 ARM64 上实现 4 路（NEON 128 bit），在 x86 上实现 8 路（AVX2 256 bit）与 16 路（AVX-512 512 bit）。

3.5 运行时调度与安全回退

为保证工程“能正常运作”，x86 平台在编译期借助 GCC 的 target attribute 为每个优化函数单独开启对应指令集，运行期通过 `__builtin_cpu_supports` 探测 `ssse3 / avx2 / avx512f+bw`，能力缺失时逐

级回退（AVX-512→AVX2→SSSE3→标量）；mb16 在无 AVX-512 的机器上自动拆为两批 8 路执行。

ARM64 的 NEON 为架构标配，直接启用。因此同一套源码在任意 x86-64 或 ARM64 机器上编译运行都能得到正确结果。

4 关键实现

4.1 工程结构与构建

工程采用 C11 编写、Makefile 构建，目录结构与各文件职责见表 2。Makefile 依据编译器目标三元组（gcc -dumpmachine）而非主机 uname 自动选择平台源文件，因此本机构建与 ARM64 交叉编译使用同一条命令，仅替换 CC 即可。

文件	职责
include/sm3.h	公共 API：流式 / 一次性 / 多块并行接口
src/sm3_scalar.c	标量参考实现（正确性基准、性能基线）
src/sm3_dispatch.c	流式 API、cpuid 运行时分派、安全回退
src/sm3_hybrid_x86.c	x86 单流混合：SSSE3 扩展 + GPR 压缩
src/sm3_hybrid_arm64.c	ARM64 单流混合：NEON 扩展 + GPR 压缩
src/sm3_mb8_avx2.c	x86 AVX2 多块并行 8 路
src/sm3_mb16_avx512.c	x86 AVX-512 多块并行 16 路
src/sm3_mb4_arm64.c	ARM64 NEON 多块并行 4 路
test / bench	标准向量与一致性验证 / 吞吐量基准

表 2 工程文件结构

构建与验证命令如下：

```
make          # x86-64 本机构建 sm3test / sm3bench
make test     # 标准测试向量 + 跨实现一致性
make bench    # 吞吐量基准
```

```
make CC=aarch64-none-linux-gnu-gcc # ARM64 交叉编译
```

4.2 ARM64 NEON 实现要点

NEON 没有 32 bit 循环移位单指令，ROTL 由 vshlq/vshrq/vorrq 三条指令合成；消息块的大端转换使用 vld1q_u8 配合 vrev32q_u8，对任意地址对齐均安全；多块并行的 4×4 字转置使用 vtrnq_u32 与 vcombine_u32/vget_low/high 组合完成，避免逐 lane 内存访问。单流混合版中，每组扩展仅链头一个字回退到通用寄存器计算，其余路径全部驻留 NEON 寄存器。

4.3 x86 AVX2 / AVX-512 实现要点

AVX2 版本以 __m256i 承载 8 条消息：装载时使用 _mm256_set_epi32 跨消息收集同序号字，再用 _mm256_shuffle_epi8 一次性完成 8 个 lane 的大端转换；多数函数化简为 $(x \wedge y) \vee (z \wedge (x \vee y))$ ，选择函数化简为 $z \oplus (x \wedge (y \oplus z))$ ，各节省一条指令。AVX-512 版本与 AVX2 同构扩展到 16 lane，并使用 AVX512F 的 _mm512_rol_epi32 (vprold) 单指令完成循环移位、AVX512BW 的 vpsshufb 做字节序转换，指令密度进一步提高。x86 单流混合版使用 SSSE3 128 bit 路径，其设计动机在 3.1 节已有分析：单消息压缩链本质串行，更宽的寄存器应留给块间并行使用。

5 正确性验证

5.1 标准测试向量

采用 GB/T 32905-2016 附录给出的两组官方示例向量验证，两个架构平台的全部实现输出与标准值完全一致（见表 3）。

输入消息	期望杂凑值（标准值）	验证结果
“abc”	66c7f0f4 62eedd9 d1f2d46b dc10e4e2 4167c487 5cf2f7a2 297da02b 8f4ba8e0	通过
“abcd”×16（64 字节）	debe9ff9 2275b8a1 38604889 c18e5a4d 6fdb70e5 387e5765 293dcba3 9c0c5732	通过

表 3 GB/T 32905-2016 标准测试向量验证

5.2 跨实现一致性验证

进一步以 1 MiB 确定性伪随机消息做交叉验证：标量实现、单流混合实现、多块并行实现（NEON 4 路 / AVX2 8 路 / AVX-512 十六路）的摘要必须逐一相同；同时以 7/64/1000/3/128/65536 字节混合粒度做流式分段喂入，结果须与一次性哈希一致。实测各实现输出的摘要均为 40af2a9eb610811c 57354718 290e0027 e9542ca4 77766955 550214a1 efc45345，ARM64（qemu 运行）与 x86-64 平台结果也完全相同，交叉验证全部通过（见表 4）。

验证项	内容	x86-64	ARM64(qemu)
标量 vs 单流混合	1 MiB 随机消息摘要一致	通过	通过
流式分段	6 种粒度分段 vs 一次性	通过	通过
多块并行 mb4	4 路输出逐路一致	通过	通过
多块并行 mb8	8 路输出逐路一致	通过	通过（回退）
多块并行 mb16	16 路输出逐路一致	通过	通过（回退）
跨平台	两平台摘要完全相同	通过	通过

表 4 跨实现、跨平台一致性验证结果

6 性能测试与分析

6.1 测试方法

单流吞吐：对同一份 256 MiB 缓冲区完整哈希，预热后重复 3 次取最优值，换算为 MiB/s。多块吞吐：准备 16 条各 16 MiB（ARM64 为 16 条各 2 MiB）的独立消息，分别以“逐条单流串行”为基线、以 multi-buffer 接口并行处理，总数据量恒定为 256 MiB（ARM64 为 32 MiB），比较聚合吞吐率。ARM64 数据在 qemu-user 模拟环境测得，绝对值不代表真实硬件，仅相对加速比有效。

6.2 x86-64 平台结果

x86-64 平台实测结果见表 5 与图 4。单流混合实现较标量基线提升 37%；多块并行收益显著：AVX2 8 路达 564.62 MiB/s，为标量基线的 4.72 倍；AVX-512 十六路达 760.54 MiB/s，为标量基线的 6.35 倍。

实现	耗时 (s)	吞吐率 (MiB/s)	相对标量加速比
标量 (通用寄存器)	2.138	119.72	1.00×
单流混合 (SSSE3+GPR)	1.562	163.88	1.37×
AVX2 多块 8 路	0.453	564.62	4.72×
AVX-512 多块 16 路	0.337	760.54	6.35×

表 5 x86-64 平台 SM3 吞吐率 (256 MiB 实测，取 3 次最优)

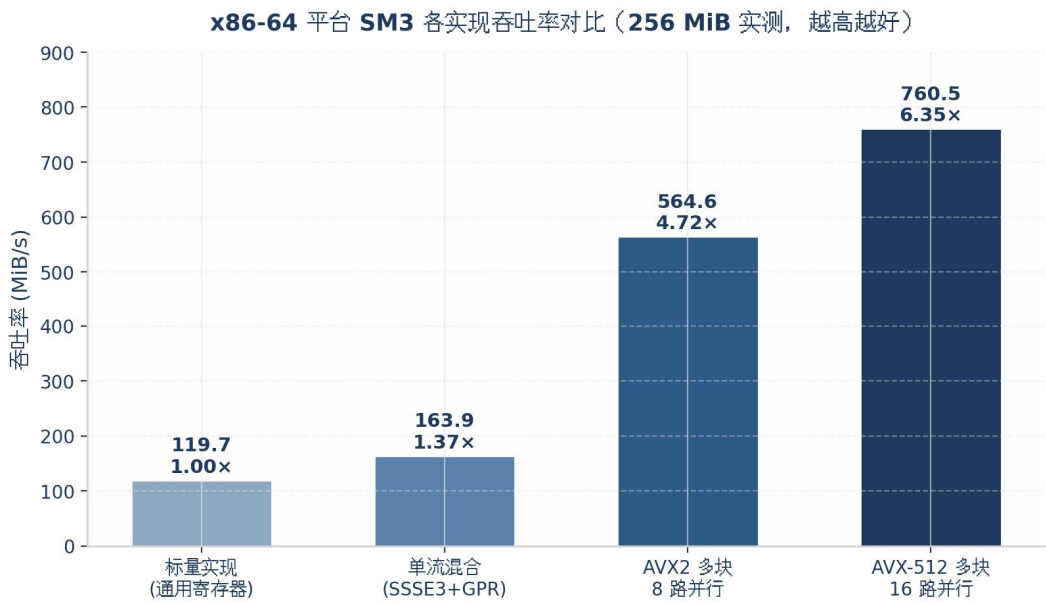


图 4 x86-64 平台各实现吞吐率对比

6.3 ARM64 平台结果

ARM64 平台（qemu-user 模拟）实测结果见表 6 与图 5。NEON 单流混合实现相对标量基线提升 97%，NEON 4 路多块并行达到标量基线的 2.92 倍。mb8 / mb16 接口在 ARM64 上按调度策略回退为单流串行（吞吐与单流持平），体现了回退机制的正确性。

实现	耗时 (s)	吞吐率 (MiB/s)	相对标量加速比
标量（通用寄存器）	1.667	19.19	1.00×
NEON 单流混合	0.848	37.74	1.97×
NEON 多块 4 路	0.572	55.97	2.92×

表 6 ARM64 平台 SM3 吞吐率（qemu-user，32 MiB 实测）

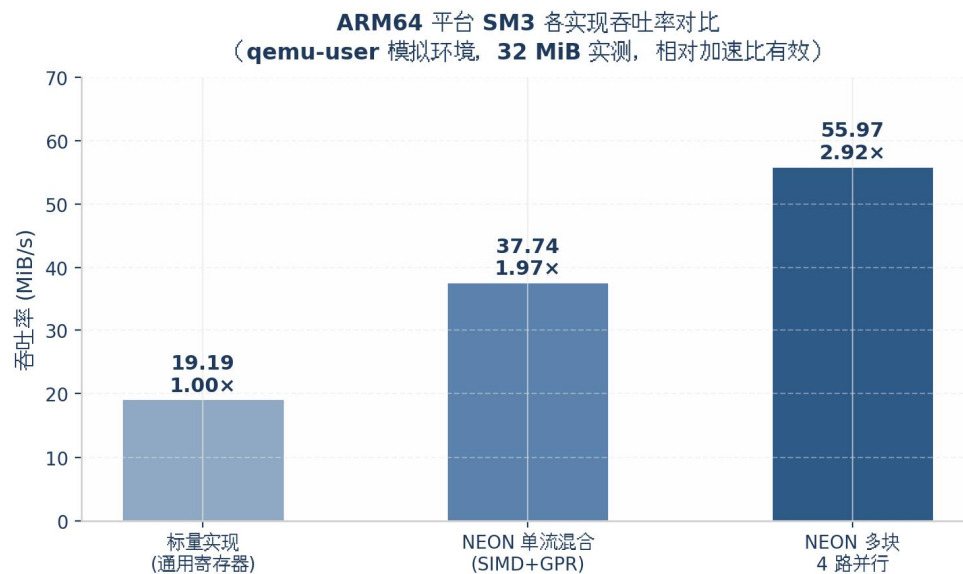


图 5 ARM64 平台各实现吞吐率对比

6.4 结果分析

(1) 单流混合收益的来源与上限。单条消息的 64 轮压缩链不可并行，SIMD 只能承接消息扩展与字节序转换，这部分约占总运算量的三分之一，与实测 1.37× (x86) 的提升幅度相符。ARM64 上标量基线指令效率较低（ROTL 需多指令合成、扩展全部串行），混合版将扩展整体向量化后收益放大到 1.97×。

(2) 多块并行近似随路数线性扩展。AVX2 8 路相对单流混合提升 3.45 $\times$ ，AVX-512 十六路相对 AVX2 提升约 1.35 $\times$ 。后者未达到理想的 2 $\times$ ，主要因为 AVX-512 重负载触发核心降频，且 16 路并行时消息扩展的寄存器驻留数据（68 个 512 bit 字）超出 L1 容量、访存压力增大。这与 SIMD 哈希优化的普遍规律一致：8–16 路已接近单核吞吐拐点。

(3) 混合策略的必要性。若盲目把压缩链也“向量化”，跨 lane 洗牌的开销会抵消并行收益；把可并行的扩展留给 SIMD、串行压缩链留给通用寄存器，再以多块并行承接宽度扩展，是本实验中投入产出比最高的组合。

7 结论与体会

本实验完成了 SM3 密码杂凑算法的软件实现与混合优化，交付的工程在 ARM64（NEON）与 x86（SSSE3 / AVX2 / AVX-512）两种指令集架构上均通过了 GB/T 32905-2016 官方测试向量与跨实现一致性验证。性能方面，x86-64 平台单流混合实现达 163.88 MiB/s（1.37 $\times$ ），AVX2 多块并行达 564.62 MiB/s（4.72 $\times$ ），AVX-512 十六路达 760.54 MiB/s（6.35 $\times$ ）；ARM64 平台 NEON 单流混合与 4 路并行分别达到标量基线的 1.97 $\times$ 与 2.92 $\times$ （qemu-user 模拟环境）。

通过本实验可以体会到：SIMD 优化的前提是对算法依赖结构的精确分析——“能并行的部分用满 SIMD 寄存器、本质串行的部分交给通用寄存器”，这正是混合优化算法的核心思想；而运行时调度与安全回退机制，则是优化代码走向工程可用的必要保障。

参考文献

- [1] 国家密码管理局. GB/T 32905-2016 信息安全技术 SM3 密码杂凑算法[S]. 2016.
- [2] Intel Corporation. Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2: Instruction Set Reference[Z]. 2024.
- [3] Arm Limited. Arm Architecture Reference Manual for A-profile architecture, Neon Programmer's Guide[Z]. 2023.

[4] 关志. GmSSL: 国密算法开源工具箱 (SM3 多路并行实现) [CP/OL]. <https://github.com/guanzhi/GmSSL>.

[5] Wang X, et al. 王小云等. SM3 杂凑算法的设计与安全性分析[J]. 密码学报, 2016.

SM3 · SIMD + GPR 混合优化

ARM64 NEON | x86 AVX2 | x86 AVX-512